Effective Unit-testing

Why now

Debuggability

Correctness

Test speed

- Development speed
- Deployment speed

Performance

Flakiness

Understandability

Future-proof

A bet

I'm going to bet that:

we will get so many Bokio users (and partners, and public API callers,) that we can't have every single request handled by our single central database.

Who wants to bet against me?

(We currently code *as if* there will only ever be one central DB)

Testing is a coding activity

Good testing starts in */src/main*, not */src/test*

What are we doing wrong in /src/main currently?

Typical backend development

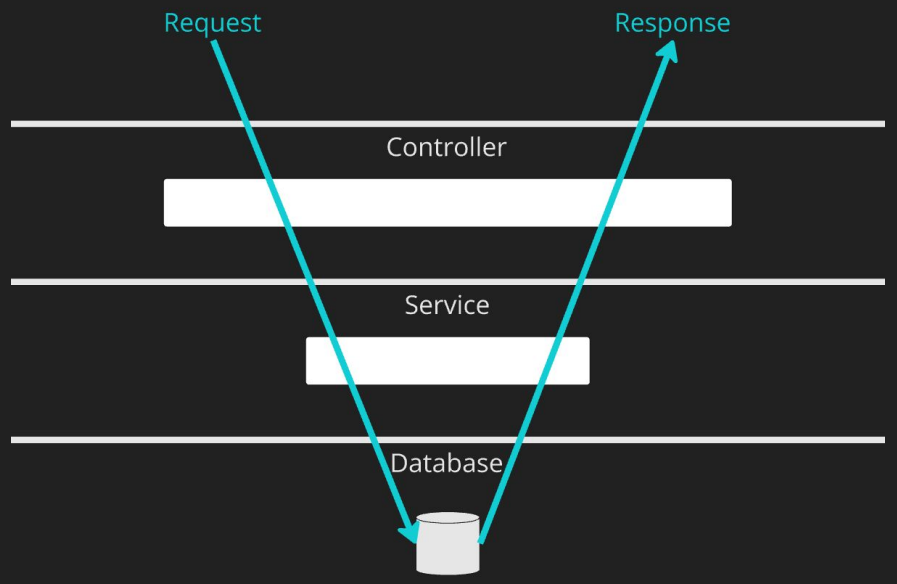
Every call will eventually hit the database

But we don't *talk about the database* when we design behaviour with external stakeholders, e.g.

"Do we calculate VAT correctly?"
- Let's fire up the database to test it!

"Do we charge the right amount for Swish?"
- Let's fire up the database to test it!

What does the database have to do with these things?



Test speed

```
WithConnectedBookkeeping_SplitSyncedTransactions_ShouldNotDuplicateOnNextSync [16 sec 491 ms] Shouldly.ShouldAssertException: avrsAfterSecondSync.Count
Shouldly.ShouldAssertException Create breakpoint : avrsAfterSecondSync.Count

Shouldly.ShouldAssertException Create breakpoint
avrsAfterSecondSync.Count
    should be
2
    but was
3
   at Bokio.Bank.Tests.Core.Controllers.EnableBankingControllerTest.WithConnectedBookkeeping_SplitSyncedTransactions_ShouldNotDuplicateOnNextS
```

The Dependency Inversion Principle (DIP)

"**Design Principles and Design Patterns**", Robert Martin, 2000.

But do we really care what some C++ programmers were doing in the 90s?

"**Hexagonal Architecture**" - Alistair Cockburn, later "**Ports & Adapters**"

Martin Fowler

"**The onion architecture**" - Jeffrey Palermo 2008

"**ECB (Entity-Control-Boundary)**" - Ivar Jacobson

"**Clean Architecture**" - Robert Martin (again) 2012

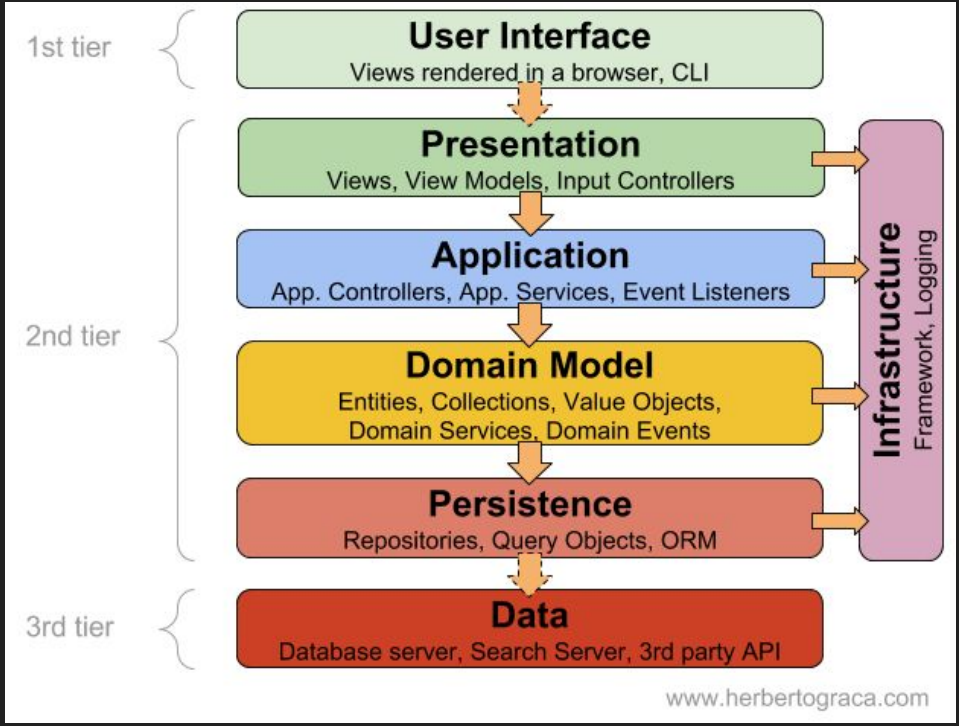
Before we dive in - What *it is* not

Not a “layered architecture” ->
You would have just coded it like this anyway.

Not “dependency injection” (although it’s related).

Certainly not a “dependency injection framework” (e.g. Spring)

Not making everything an interface: IController, IService, IRepository, etc.



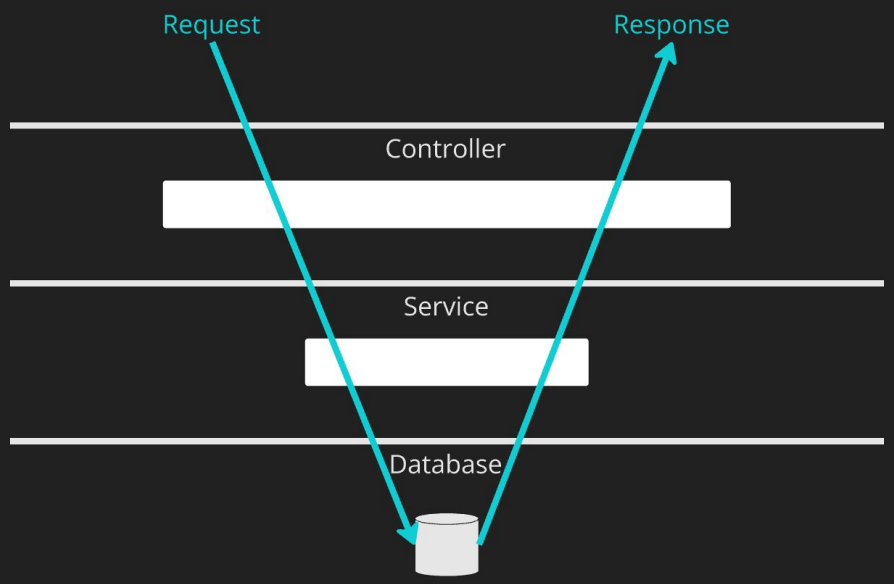
The Dependency Inversion Principle (DIP)

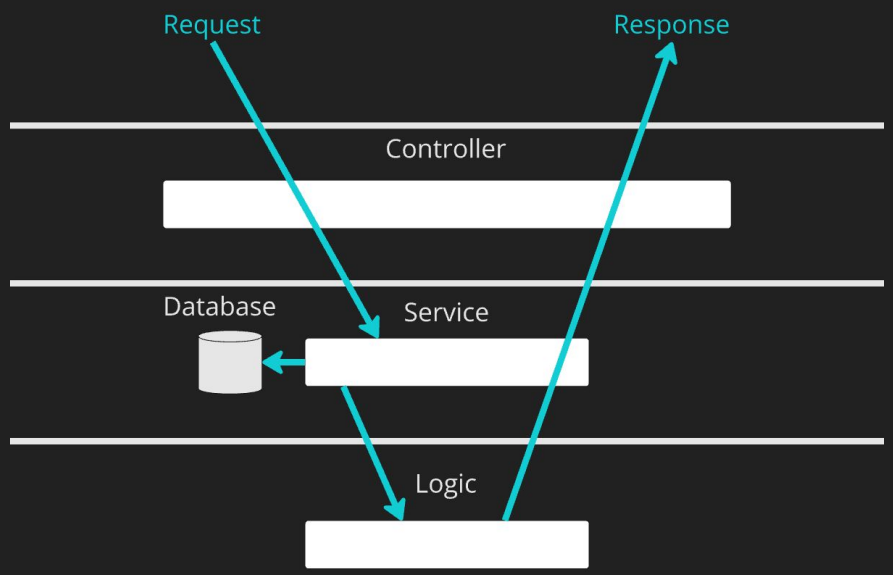
“Depend upon Abstractions. Do not depend upon concretions.”

Not very instructive.

The Dependency Inversion Principle (DIP)

“Depend upon Abstractions. Do not depend upon concretions.”





We try to hide complexity in little black boxes

We try to hide complexity in little black boxes

The database is bigger than us,

We are its black box!

By the way,

Who’s ever tried abstracting away a REST call?

IRestManager (IRestClient (IRestHelper, (ConnectionManager, (RestRunner))))

What is an abstraction? What is a concretion?

{ Abstraction / business-logic / deterministic / pure function / things you discuss with your PO / can be true independently of a computer / essential complexity }

{ Concretion / IO / Computer-related / nondeterministic / Database / Network / will sometimes fail (by definition) / Flaky / incidental complexity }

What is an abstraction? What is a concretion?

Are any of these more abstract or more concrete than the others?

User, Database, Buffered, Postgres, VAT

What about

UserDatabase, BufferedDatabase

Dive into some actual code

My current struggle:

- Identifying duplicate imported transactions
- “Why I’m finding this hard”
- AutoVerificationHelper.CompareAndCreateTransactionsByCountingGroup
- AutoVerificationHelperTest

Previous work:

- Swish billing:
- “Why I found this easy”
- SveaProductHistoryService
- SveaProductHistoryServiceTest

DipApp:

- A fake example for teaching

Summary

Good testing is an activity for src/main, not src/test

Fix code in src/main using dependency inversion

Identify and separate your logic from your dependencies

Push IO up the call chain; push logic down the call chain

Replace IO call with *focussed* and *well-typed* interfaces

You know it's working if you can *new()* and run your business logic without touching a database or a network connection